# Repair Bst

AlgoExpert · Hard · Binary Search Trees

# The problem

- Exactly two nodes' values were swapped; swap them back in place.
- Iterative in-order finds the two out-of-order nodes.

# Intuition

- An in-order traversal of a BST is sorted; the two swapped nodes show up as order violations.

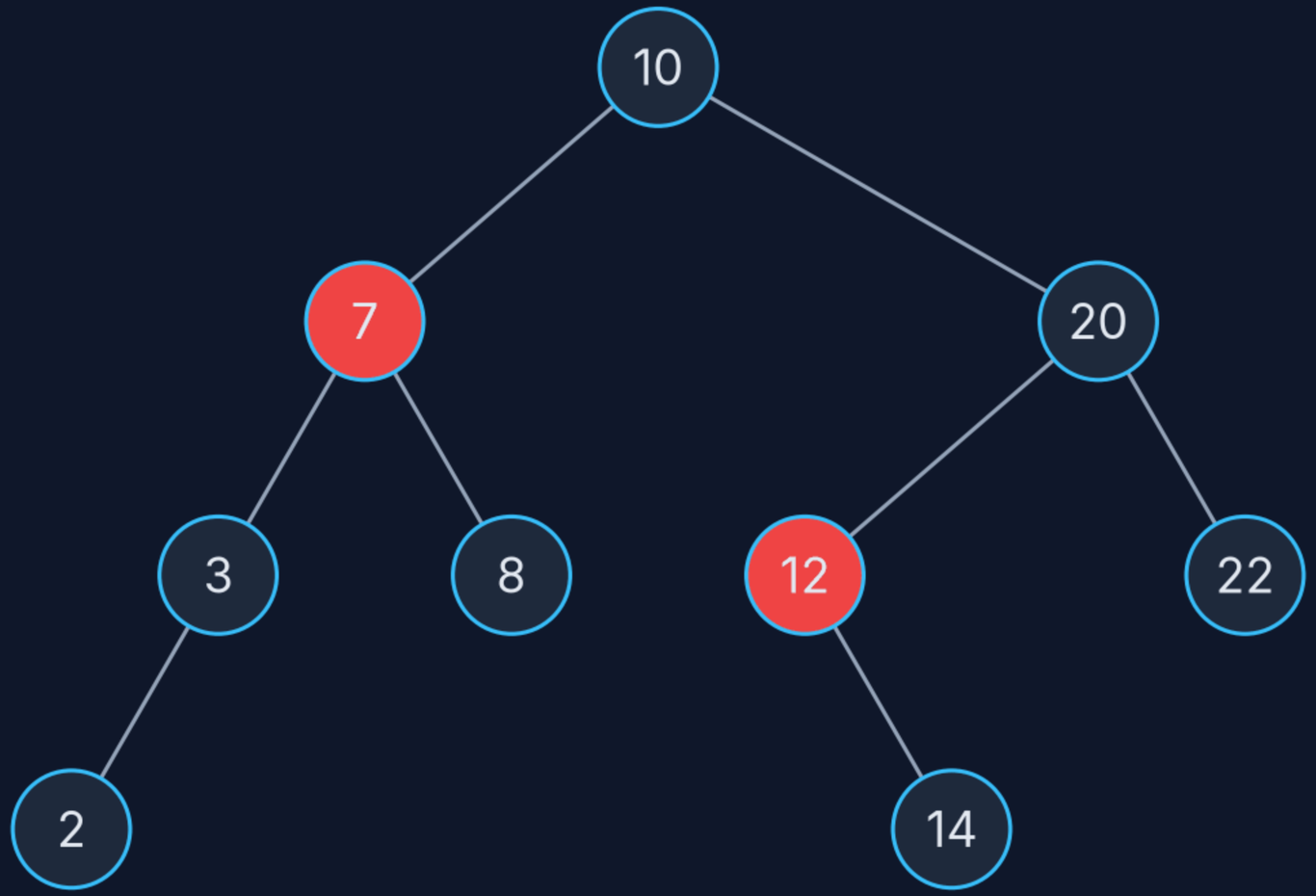
# Brute force vs optimal

- Copy in-order to array, sort, rewrite → O(n) space
- In-order with prev pointer, catch the two violations → O(h) space
- Morris traversal → O(1) space

# Approach

- 1. In-order traverse tracking the previous node
- 2. The first violation gives the first node
- 3. The last violation gives the second
- 4. Swap their values

# Worked example

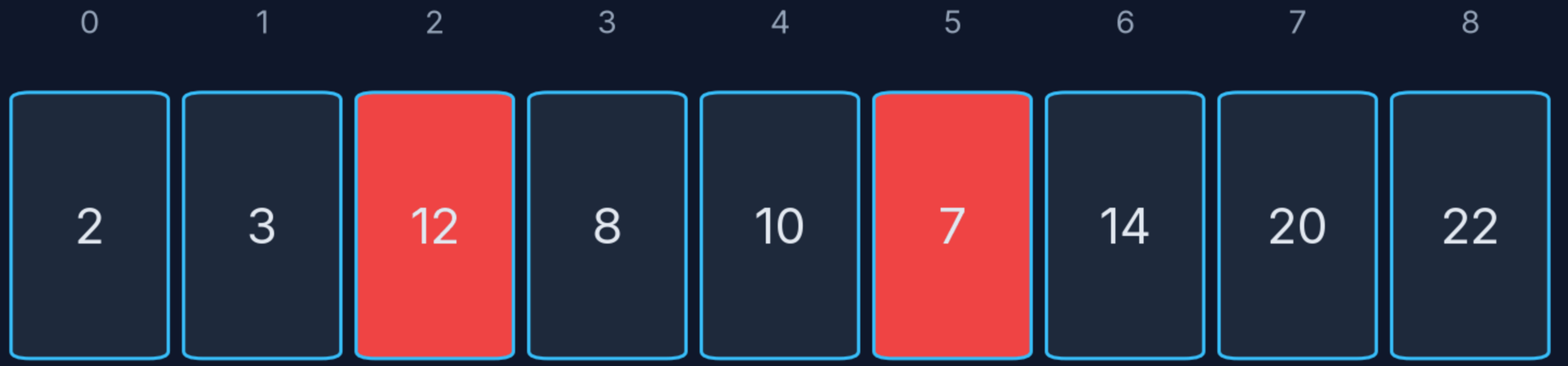



two values swapped

A BST with two swapped values

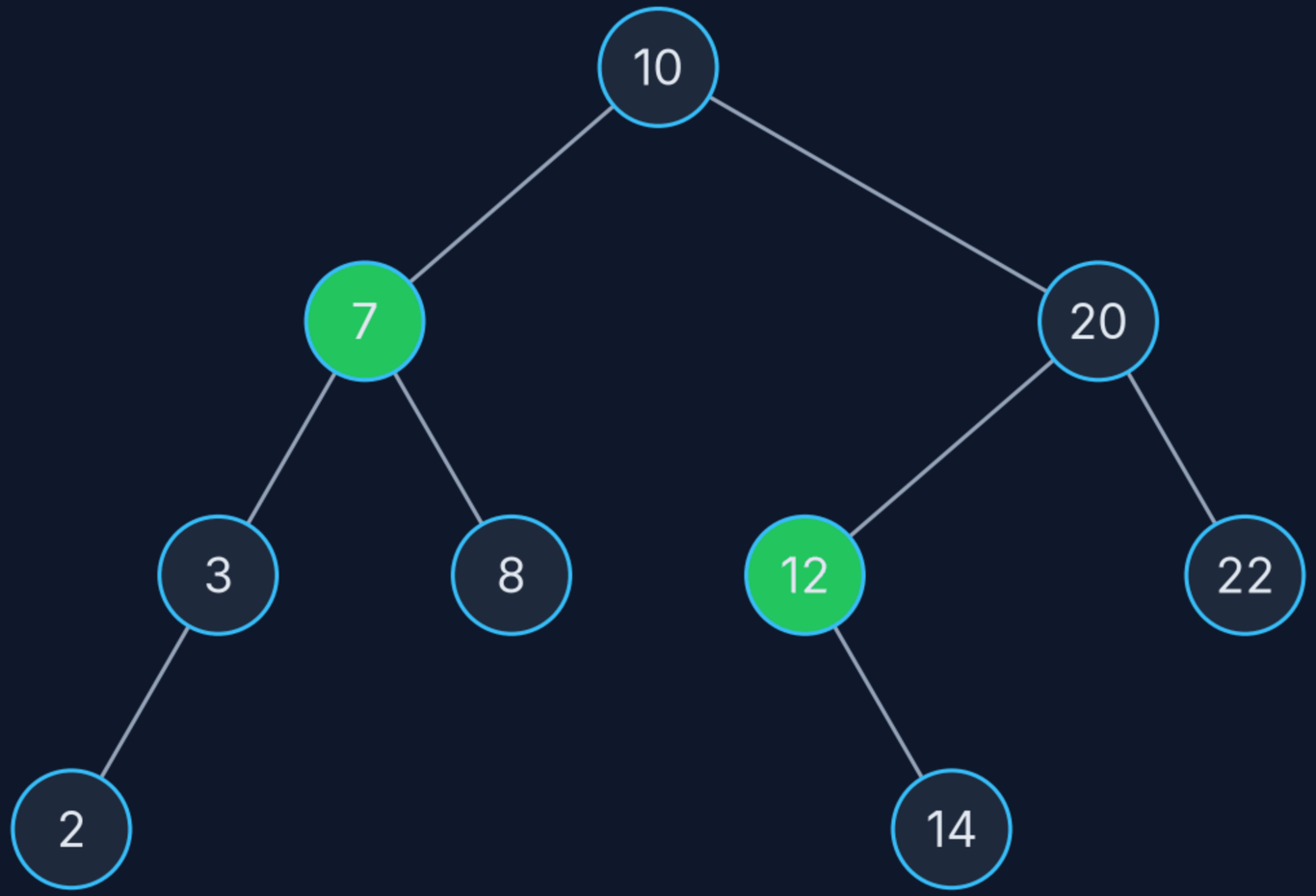
# Worked example

in-order traversal




In-order should be sorted: drops at [12, 10] > [8, 7]

# Worked example




in-order

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| 2 | 3 | 7 | 8 | 10 | 12 | 14 | 20 | 22 |

Swap them back → valid BST

# Java solution

```java
public class RepairBst {
    public static BST repairBst(BST tree) {
        BST first = null, second = null, prev = null, cur = tree;
        ArrayDeque<BST> st = new ArrayDeque<>();
        while (cur != null || !st.isEmpty()) {
            while (cur != null) { st.push(cur); cur = cur.left; }
            cur = st.pop();
            if (prev != null && prev.value > cur.value) { if (first == null) first = prev; second = cur; }
            prev = cur;
            cur = cur.right;
        }
        int t = first.value; first.value = second.value; second.value = t;
        return tree;
    }
}
```

# Complexity

- Time: O(n)
- Space: O(h)

# Edge cases & pitfalls

- Adjacent swap → only one drop
- First = prev at first drop, second = cur at last drop
- Swap values, not nodes

# Interview tips & follow-ups

- Mention Morris traversal for O(1) space

# Recap

- An in-order traversal of a BST is sorted; the two swapped nodes show up as order violations.
- Complexity: O(n)